

Autómatas, Teoría de Lenguajes y Compiladores

Proyecto Especial

Instituto Tecnológico de Buenos Aires



Grupo G 108

Integrantes:

- Fernández Flores, Valentina 64142
- Priolo, Julia
- Rodríguez, Olivia

Fecha de Entrega: 28/11/2025

2025

Tabla de contenidos

1. Introducción.....	3
2. Modelo Computacional.....	3
2.1. Dominio.....	3
2.2. Lenguaje.....	3
3.1. Frontend.....	5
3.2. Backend.....	5
3.3. Adicionales.....	6
3.4. Dificultades Encontradas.....	6
4. Futuras Extensiones.....	6
5. Conclusiones.....	7
6. Bibliografía.....	7

1. Introducción

En el presente informe se documenta el desarrollo y la implementación de un lenguaje específico de dominio (DSL) orientado a la generación automática de gráficos a partir de datos en archivos csv. El objetivo principal es permitir que usuarios sin conocimientos HTML o JavaScript puedan generar visualizaciones declarando únicamente el tipo de gráfico, los datos a utilizar y algunas propiedades esenciales.

El DSL desarrollado soporta múltiples tipos de gráficos, operaciones de filtrado agregación y composición de fuentes de datos CSV. La implementación se basa en herramientas de análisis léxico y sintáctico como Flex y Bison, junto con un backend en lenguaje C para el procesamiento de datos y generación de código. La salida final consiste en un archivo HTML que emplea la librería Chart.js¹ para el renderizado interactivo en páginas web.

2. Modelo Computacional

El desarrollo del lenguaje propuesto se estructuró en torno a dos decisiones principales: el dominio del problema y el lenguaje diseñado para abordarlo. Ambos elementos definen el alcance y el comportamiento del compilador.

2.1. Dominio

El dominio del proyecto se centra en la construcción de un lenguaje que permita describir, de manera declarativa, la generación de gráficos web a partir de archivos CSV. Este campo surge de la necesidad de simplificar un proceso que normalmente requiere combinar múltiples tecnologías como HTML, CSS, JavaScript y librerías específicas de visualización, las cuales pueden resultar complejas de manejar, especialmente para usuarios sin experiencia en desarrollo web.

El trabajo busca abstraer por completo ese proceso técnico para que el usuario solo deba especificar qué datos quiere visualizar y de qué forma, mientras que el compilador se ocupa de resolver la manipulación de datos, el armado de estructuras intermedias y la producción final de un archivo HTML válido.

La estructura del lenguaje evita anidamientos complejos y sigue un formato lineal que refleja el flujo lógico del dominio: cargar, procesar y visualizar. Las palabras clave se definieron tomando conceptos reconocibles como source, from, char, type; lo que permite una relación directa entre la descripción escrita y las operaciones reales que realizará el compilador.

2.2. Lenguaje

Como se mencionó anteriormente, el lenguaje de entrada está basado en un DSL diseñado específicamente para la descripción declarativa de gráficos lo que permite al usuario definir visualizaciones de datos de manera intuitiva y estructurada.

Construcción	Descripción
source	Declara una fuente de datos basada en un archivo CSV o en otra source.

¹ Documentación oficial de Chart.js: <https://www.chartjs.org/>

from	Especifica la ubicación del CSV o el identificador de una source previa.
filter	Aplica condiciones lógicas sobre columnas (>, <, ==, AND, OR...).
project	Selecciona un subconjunto de columnas.
chart	Declara el título de un gráfico e incluye todas sus propiedades.
type	Define el tipo de gráfico (bar, pie, scatter, donut, line).
x / y	Asignan las variables a los ejes.
colors	Lista de colores hexadecimales.
orientation	Vertical u horizontal (para bar charts).
id	Identificador único del gráfico.
range	Define rangos para los ejes.

Tabla 1: Construcciones del DSL.

Cada elemento del DSL se asocia con una estructura jerárquica como se muestra a continuación en el caso de una definición de fuente de datos y un gráfico de barras

```
source mi_fuente = from "datos.csv"
filter variable1 > 10
project [variable1, variable2];

chart "Mi Gráfico" type bar:
from "datos.csv",
x = variable1,
y = variable2,
orientation = vertical,
colors = [#4CAF50, #2196F3],
id = "grafico_ejemplo";
```

Código 1 : Ejemplo de entrada

El primer campo obligatorio es chart con su type (por ejemplo, bar, scatter, pie), ya que sin él no se puede generar ninguna visualización. Lo mismo ocurre con el campo from, que especifica la fuente de datos y es esencial para que el compilador sepa de dónde obtener la información.

A nivel sintáctico, se implementaron las siguientes representaciones:

- Definiciones de fuentes de datos con filtros y proyecciones (FROM con FILTER y PROJECT).
- Selección de tipos de gráficos (BAR, SCATTER, PIE, DONUT, LINE).
- Especificación de ejes X e Y, incluyendo expresiones matemáticas.
- Configuración de colores, orientaciones e identificadores únicos.
- Operaciones de agregación (como AVERAGE, MIN, MAX, COUNT, SUM).

- Alias para expresiones y columnas calculadas.

El lenguaje de salida se corresponde con un subconjunto de HTML y JavaScript, donde se basa principalmente el diseño para la renderización interactiva utilizando la librería Chart.js sobre una estructura de página web. La generación de este código se realiza una vez que se valida la sintaxis del DSL de entrada y se construye el árbol de sintaxis abstracta (AST) mediante las herramientas Flex y Bison.

3. Implementación

Para la implementación del compilador se utilizó como base del proyecto [Flex-Bison-Compiler](#) (sobre la rama development), que facilitó el uso de las herramientas Flex (para el análisis léxico) y Bison (para el análisis sintáctico), como ya se describió previamente.

3.1. Frontend

Para desarrollar el frontend, el énfasis estuvo principalmente en la definición de una gramática formal que representara el lenguaje deseado para la especificación de gráficos. De esta manera se definieron como símbolos terminales: strings, enteros, números decimales, colores en formato hexadecimal, identificadores y diversos tipos de tokens como keywords ('source', 'chart', 'from', 'filter'), operadores aritméticos y de comparación, tipos de gráficos como 'pie', 'bar', 'line', etc.

Por otro lado, se definieron como no terminales las estructuras de más alto nivel: las declaraciones de 'source_decl' y 'chart_decl' que constituyen los statements principales del lenguaje, las cláusulas 'FROM', 'FILTER', 'PROJECT' para la definición de fuentes de datos, las expresiones aritméticas y las funciones de agregación que pueden utilizarse en el eje Y de los gráficos, y el símbolo inicial 'program' del cual se deriva la estructura completa del programa DSL.

Desde el frontend fue posible validar muchas estructuras y descartar otras tantas consideradas inválidas dentro del lenguaje. Entre las validaciones sintácticas realizadas se encuentran: la detección de identificadores de source duplicados, la verificación de que las referencias a sources existan cuando se utilizan en composiciones, y la validación de que un chart tenga al menos una fuente de datos asociada. Sin embargo hubo un par de casos que, debido a la estructura de la propia gramática y sus producciones, se debieron postergar para validaciones semánticas del backend, tales como la verificación de que las columnas referenciadas en filtros, proyecciones y expresiones existan realmente en los archivos CSV, o que los tipos de datos sean compatibles en las operaciones aritméticas.

3.2. Backend

El desarrollo del backend se estructuró en cuatro módulos especializados: procesamiento de CSV, procesamiento de datos, procesamiento de gráficos y generación de código.

El procesador de CSV se encarga de leer y parsear archivos desde el sistema de archivos, manejando comillas, comas dentro de valores y encabezados. El módulo de procesamiento de datos implementa las operaciones semánticas sobre los datos CSV: aplicación de filtros con operadores de comparación, proyección de columnas validando su existencia, evaluación recursiva de expresiones aritméticas sobre el AST, y funciones de agregación (promedio, suma, mínimo, máximo, conteo) para generar gráficos con datos resumidos. El procesador de gráficos prepara los datos finales para

visualización, manejando múltiples sources (desde archivos CSV o mediante identificadores) y combinándolas cuando es necesario.

Y finalmente, el módulo de generación de código produce el HTML y JavaScript final utilizando Chart.js. La generación se realiza en dos fases: primero se procesan todas las sources creando un mapa de identificadores a datos procesados (manejando sources desde archivos y sources compuestas), y luego se procesan los charts generando el código JavaScript con Chart.js. Durante el procesamiento se realizan validaciones semánticas adicionales: verificación de existencia de archivos CSV, validación de columnas referenciadas y verificación de referencias válidas en sources compuestas.

3.3. Adicionales

En el archivo README.md del proyecto se documenta el proceso para ejecutar tests individuales y visualizar su salida. Específicamente, se especifica el comando y la ruta necesaria para ejecutar un test específico y guardar su salida HTML en un archivo. Para visualizar el contenido del archivo HTML generado, puede utilizarse cualquier navegador web. Alternativamente, es posible emplear el visor en línea de W3Schools².

3.4. Dificultades Encontradas

Una de las principales dificultades encontradas durante el desarrollo del backend fue el manejo de la memoria y la liberación de datos CSV. Dado que los datos pueden provenir de diferentes fuentes (archivos CSV leídos directamente, sources predefinidas mediante identificadores, o combinaciones de múltiples sources), se debió implementar una lógica para determinar cuándo liberar la memoria y cuándo mantener referencias a datos compartidos. Esto fue particularmente crítico en el caso de sources compuestas, donde los datos procesados pertenecen a un mapa central y no deben liberarse hasta que todas las referencias hayan terminado de utilizarlos.

Otra dificultad surgió durante la implementación de la composición de sources y el orden de procesamiento. Para permitir que una source pueda construirse a partir de otra source previamente definida, fue necesario implementar un sistema de dos pasadas en el generador de código: primero se procesan todas las sources que provienen directamente de archivos CSV, y luego se procesan las sources compuestas que dependen de sources ya procesadas. Este enfoque fue necesario para garantizar que las dependencias entre sources se resuelvan correctamente y que el procesamiento se realice en el orden adecuado.

4. Futuras Extensiones

Si bien el compilador implementado cumple con los objetivos propuestos, existen algunas oportunidades de expansión que permiten aumentar el poder expresivo del DSL. Entre las posibles extensiones se encuentran:

- Nuevos tipos de gráficos: Incorporar soporte para gráficos adicionales como radar, area charts, histogramas o boxplots.

² Página web de W3Schools: https://www.w3schools.com/html/tryit.asp?filename=tryhtml_intro

- Operaciones avanzadas sobre fuentes de datos: Se podría agregar funcionalidad para realizar Joins entre múltiples CSV por clave o agrupamientos con funciones avanzadas como medianas, percentiles, desviación estándar.
- Configuraciones de estilo avanzadas: Se podría aprovechar más capacidades de [Chart.js](#) agregando animaciones configurables, temas predefinidos, gráficos 3D, control fino de ejes, tipografías y layouts.
- Integración con otros formatos de datos: Extender el soporte más allá de CSV para incluir bases de datos SQL (como PostgreSQL) o archivos JSON/XML, permitiendo consultas más complejas y validaciones de esquemas.
- Exportación y persistencia de gráficos: Agregar opciones para exportar los gráficos generados a formatos como PDF, PNG o SVG, facilitando su uso en informes o presentaciones sin depender del navegador.

5. Conclusiones

En conclusión, se desarrolló un compilador que cumple con el objetivo principal: permitir que un usuario genere gráficos web a partir de archivos CSV mediante un DSL simple y declarativo. Se implementó un frontend con Flex y Bison que construye correctamente el AST, y un backend capaz de procesar datos, aplicar filtros y proyecciones, verificar referencias y generar un archivo HTML funcional. A pesar de las dificultades que se lograron resolver de manera efectiva, se obtuvo un lenguaje funcional que automatiza la creación de visualizaciones y deja una base para posibles futuras mejoras.

6. Bibliografía

-  (2025-08-21, v3.0.8) Proyecto Especial
-  (2025-08-21, v0.6.0) Proyectos Anteriores
-  (2025-09-30, v1.0.1) Especificación
-  (2025-03-13, v0.1.0) Modelo Computacional
-  (2025-09-03, v1.0.0) Análisis Léxico
-  (2024-05-08, v0.1.0) Análisis Sintáctico
-  (2024-09-25, v0.2.0) Análisis Semántico
-  (2024-05-16, v0.1.0) Generación de Código